

OrangeOS v1.0

俄罗斯方块核心代码 与答辩讲解

主循环 | 键盘中断 | PIT计时 | 方块编码 | 碰撞检测 | 旋转 | 消行 | VGA绘制

核心文件: D:\Oranges\kernel\tetris32.asm
单文件代码导读

为什么选择俄罗斯方块作为“印象最深”部分

俄罗斯方块的优势是核心逻辑集中在一个汇编文件中，现场容易定位；同时它并非与操作系统无关，而是把IRQ1键盘输入、PIT时钟、VGA显存、事件循环、状态机和自动回归连接起来，适合讲“如何在没有标准库和图形框架的裸机环境中完成一个持续交互应用”。

推荐回答

我印象最深的是在OrangeOS中实现俄罗斯方块。普通应用开发可以直接使用窗口、定时器和键盘接口，但在这个系统里，这些基础能力都需要自己提供：键盘动作来自IRQ1扫描码，自动下落由PIT tick驱动，画面直接写入0xB8000的VGA文本显存，游戏逻辑则用一个10×18棋盘数组和方块坐标表完成。实现过程中，我把中断处理和游戏逻辑分开：中断只记录动作，主循环再进行碰撞检测、状态更新和重绘。这一部分让我最直观地理解了操作系统怎样为上层应用提供输入、时间和显示抽象。

完整执行链

Shell输入tetris → tetris_run32初始化 → sti开启中断 → IRQ1写入action → 主循环验证并提交动作 → PIT tick触发自动下落 → 锁定/消行/生成新块 → 写VGA显存 → Q/Esc退出并返回Shell

代码导航

代码段	行号	职责
tetris_run32	14-136	初始化与主循环
tetris_keyboard32	139-172	扫描码转动作
tetris_step_down32 / lock	174-206	下落与锁定
tetris_spawn32	208-231	伪随机生成
tetris_can_place32	233-278	碰撞检测
shape offset	280-296	方块坐标索引
clear lines	298-338	满行消除
draw	340-418	VGA绘制
tetris_shapes	487-502	7类×4旋转坐标表

1 初始化与事件主循环

入口位于kernel/tetris32.asm:14。进入游戏后设置活动、退出、Game Over和动作状态，清零分数、行数以及10×18棋盘，再用当前tick初始化随机种子。

```
tetris_run32:
    pushad
    mov byte [tetris_active],1
    mov byte [tetris_exit],0
    mov byte [tetris_game_over],0
    mov byte [tetris_action],0
    mov dword [tetris_score],0
    mov dword [tetris_lines],0
    mov edi,tetris_board
    xor eax,eax
    mov ecx,BOARD_W*BOARD_H
    rep stosb
    call timer_get_ticks32
    mov [tetris_seed],eax
    call tetris_spawn32
    call tetris_draw32
    sti
```

主循环先读取并清零tetris_action，按1-5分派左、右、软降、旋转和硬降。没有动作时检查PIT时间，最后执行hlt等待下一次中断。

```
.loop:
    cmp byte [tetris_exit],0
    jne .leave
    mov al,[tetris_action]
    mov byte [tetris_action],0
    cmp al,1 / je .left
    ...
    call timer_get_ticks32
    sub edx,[tetris_last_tick]
    cmp edx,30
    jb .sleep
    call tetris_step_down32
    call tetris_draw32
.sleep:
    hlt
    jmp .loop
```

为什么用hlt

游戏没有输入且未到更新时间时不需要持续占满CPU。hlt让处理器等待PIT或键盘中断，被唤醒后再检查动作和时间。它仍是简化事件循环，而不是具有阻塞队列的用户进程。

现场讲解词

主循环把输入事件和时间事件统一处理。游戏状态只在主循环修改，中断只写动作标志，从而避免在IRQ1里执行碰撞检测和全屏重绘。

2 键盘中断与动作队列

tetris_keyboard32位于139行，由全局IRQ1键盘处理器优先调用。若tetris_active=0返回0，让按键继续传给监视器、GUI或Shell；若游戏活动则消费按键并返回1。

扫描码	按键	动作值	效果
0x4B	Left	1	左移
0x4D	Right	2	右移
0x50	Down	3	软降
0x48	Up	4	顺时针旋转
0x39	Space	5	硬降
0x10/0x01	Q/Esc	退出标志	返回Shell

```
tetris_keyboard32:
    cmp byte [tetris_active],0
    je .unused
    cmp al,0x4B
    je .left
    ...
.left:    mov byte [tetris_action],1
.right:  mov byte [tetris_action],2
.down:   mov byte [tetris_action],3
.rotate: mov byte [tetris_action],4
.drop:   mov byte [tetris_action],5
.quit:   mov byte [tetris_exit],1
```

为什么不在中断里直接移动方块

- 中断处理应尽量短，避免延迟PIT等其他中断。
- 碰撞检测和重绘工作量更大，更适合主循环。
- 集中修改游戏状态可以减少中断和主循环同时操作同一数据的风险。
- 当前只有一个字节动作槽，极短时间内多个按键可能覆盖；更完善的设计可以使用环形事件队列。

现场讲解词

IRQ1把硬件扫描码转换成应用级动作。返回1代表俄罗斯方块拥有当前键盘焦点，Shell不会同时收到方向键。这是一个简化的输入焦点和事件投递机制。

3 PIT计时、自动下落与硬降

PIT约100 Hz，即每tick约10 ms。主循环要求当前tick与上次tick相差至少30，因此方块约每0.3秒自动下降一格。

100 tick/s → 30 tick/update → 约0.30 s/格

```
call timer_get_ticks32
mov edx,eax
sub edx,[tetris_last_tick]
cmp edx,30
jb .sleep
mov [tetris_last_tick],eax
call tetris_step_down32
```

单步下降使用“试探-验证-提交”：先检查y+1是否合法，合法才增加piece_y；失败则锁定。

```
mov eax,[piece_x]
mov ebx,[piece_y]
inc ebx
mov dl,[piece_rot]
call tetris_can_place32
test eax,eax
jz tetris_lock32
inc dword [piece_y]
```

硬降循环不断试探下一行，成功就下移并加2分，失败时立即锁定。

```
.drop_loop:
; trial y+1
call tetris_can_place32
test eax,eax
jz .drop_lock
inc dword [piece_y]
add dword [tetris_score],2
jmp .drop_loop
.drop_lock:
call tetris_lock32
```

课程联系

PIT中断维护系统时钟，游戏读取统一tick而不是自己占用硬件定时器。它体现了操作系统把硬件时间源抽象为多个内核模块都能使用的时间服务。

4 方块数据结构与旋转

棋盘尺寸是10×18，tetris_board使用180字节，每格0表示空、1表示已经锁定。活动方块不立即写进棋盘，而是单独保存type、rotation、x、y，绘制时叠加。

变量	含义
piece_type	0-6, 对应I/O/T/S/Z/J/L
piece_rot	0-3, 四种旋转状态
piece_x / piece_y	方块参考位置
tetris_board[180]	已锁定格子
trial_x/y/rot	候选移动或旋转

tetris_shapes把每种方块、每种旋转写成4组(dx,dy)。容量是7×4×4×2=224字节。

索引 = ((piece_type × 4) + rotation) × 4 + block) × 2

```
movzx eax,byte [piece_type]
movzx ebx,byte [piece_rot]
shl eax,2
add eax,ebx
shl eax,3
lea eax,[eax+ecx*2]
movsx esi,byte [tetris_shapes+eax] ; dx
movsx edi,byte [tetris_shapes+eax+1] ; dy
```

旋转时先计算(piece_rot+1)&3，再用碰撞检测验证。合法才提交piece_rot，非法就保持原角度。

```
mov dl,[piece_rot]
inc dl
and dl,3
call tetris_can_place32
test eax,eax
jz .sleep
mov [piece_rot],dl
```

局限

当前旋转没有wall kick，靠近墙壁时只要新姿态越界就拒绝旋转。现代Tetris通常会尝试左右偏移后再次验证。

5 碰撞检测：核心算法

tetris_can_place32从233行开始，是最值得现场展示的函数。输入EAX=x、EBX=y、DL=rotation，遍历方块的4个格子，全部合法返回1。

```
.cell:
    call tetris_trial_offset32
    mov eax,[trial_x]
    add eax,esi          ; absolute x
    cmp eax,0
    jl .blocked
    cmp eax,BOARD_W
    jge .blocked
    mov ebx,[trial_y]
    add ebx,edi          ; absolute y
    cmp ebx,BOARD_H
    jge .blocked
    test ebx,ebx
    js .next             ;
    imul ebp,ebx,BOARD_W
    add ebp,eax
    cmp byte [tetris_board+ebp],0
    jne .blocked
    ...
    mov eax,1
```

它检查三类约束

约束	条件	失败结果
左右边界	$0 \leq x < 10$	blocked
底部边界	$y < 18$	blocked
棋盘占用	$\text{board}[y \times 10 + x] = 0$	blocked

为什么允许 $y < 0$

方块在顶部生成或旋转时，个别格子理论上可能暂时位于可见棋盘上方。代码跳过负y的棋盘访问，但仍检查x和底部，避免数组负索引。

现场讲解词

所有左移、右移、下降和旋转都不直接修改状态，而是先调用同一个碰撞检测函数验证候选状态。这样把边界规则集中在一处，避免不同动作使用不同判断标准。

6 锁定、生成新块与Game Over

当方块不能继续下降时，tetris_lock32遍历4格，把活动方块写入棋盘数组，然后消行并生成新方块。

```
tetris_lock32:
    xor ecx,ecx
.block:
    call tetris_shape_offset32
    ; board_index=(piece_y+dy)*10+(piece_x+dx)
    mov byte [tetris_board+ebx],1
    inc ecx
    cmp ecx,4
    jb .block
    call tetris_clear_lines32
    call tetris_spawn32
```

生成函数使用线性同余公式更新种子，再对7取余得到方块类型：

```
seed = seed × 1103515245 + 12345; piece_type = seed mod 7
```

```
imul eax,eax,1103515245
add eax,12345
mov [tetris_seed],eax
xor edx,edx
mov ecx,7
div ecx
mov [piece_type],dl
mov byte [piece_rot],0
mov dword [piece_x],3
mov dword [piece_y],0
```

生成后立即调用碰撞检测。如果出生位置已被棋盘占用，设置tetris_game_over=1。

随机性的准确说法

这是用PIT tick作为初始种子的伪随机数，不是密码学随机，也没有现代Tetris常见的7-bag均匀洗牌。因此短期内可能连续产生同一种方块。

7 满行消除与计分

tetris_clear_lines32从棋盘底部第17行向上扫描。每行10个格子全部非0即为满行。

```
mov ebx,BOARD_H-1
.row:
    xor ecx,ecx
    imul esi,ebx,BOARD_W
.scan:
    cmp byte [tetris_board+esi+ecx],0
    je .not_full
    inc ecx
    cmp ecx,BOARD_W
    jb .scan
```

发现满行后，从该行开始把上方每一行复制到下一行，再把最顶行清零。

```
.shift:
    ; dst=row, src=row-1
    mov ecx,BOARD_W
    rep movsb
    dec edx
    jmp .shift
.clear_top:
    mov edi,tetris_board
    xor eax,eax
    mov ecx,BOARD_W
    rep stosb
    inc dword [tetris_lines]
    add dword [tetris_score],100
    jmp .row
```

为什么消行后重新检查同一行

上方内容下移后，当前行可能仍是满行。如果马上ebx--会漏掉连续满行，所以代码跳回.row再次扫描当前行；只有未满足时才向上一行。

计分规则

- 每消除一行加100分。
- 硬降每成功下降一格加2分。
- 软降当前不额外加分。
- 计分规则是教学简化，没有连击、等级和多行倍率。

8 VGA文本模式绘制与自动测试

tetris_draw32直接写0xB8000。屏幕80×25，每个单元2B：低字节字符，高字节颜色。它先清屏，再绘制标题、提示、分数、行数、边框、固定棋盘和活动方块。

对象	字符/属性
边框	' ', 白色
固定格子	'#', 青色
活动方块	'@', 黄色
Game Over	红色文字
背景	空格与颜色属性

```
mov edi,VGA
mov ecx,80*25
mov ax,0x0120
rep stosw
...
cmp byte [tetris_board+edx],0
mov ax,0x0B23 ; '#' fixed block
...
mov word [edx],0x0E40 ; '@' active block
```

退出时cli、清除tetris_active、清空VGA、恢复Shell光标，再popad/ret。

自动回归

Makefile中的test-tetris启动QEMU，通过sendkey输入tetris和方向键、空格、Q；日志必须同时出现TETRIS READY和OrangeOS>，证明应用启动、按键消费以及返回Shell路径有效。

测试局限

当前测试不逐格验证棋盘，也不自动构造满行；它更接近启动与交互冒烟测试。可改进为输出结构化棋盘状态、固定随机种子，并自动验证旋转、碰撞和消行结果。

9 答辩问答与代码展示顺序

两分钟代码展示

时间	位置	讲解重点
0:00-0:25	14-59	初始化、PIT与hlt主循环
0:25-0:50	139-170	IRQ1只写动作，焦点返回1
0:50-1:20	233-278	四格统一碰撞检测
1:20-1:40	298-338	自底向上消行和同一行复查
1:40-2:00	340-418	直接写VGA文本显存

高频问题

- 它为什么能体现操作系统？输入来自IRQ1、时间来自PIT、显示直接写VGA，应用依赖内核提供的硬件抽象。
- 为什么中断只记录action？缩短中断时间，把复杂状态更新放回主循环。
- 如何避免方块越界？所有动作先调用tetris_can_place32，检查左右、底部和棋盘占用。
- 旋转如何实现？每个方块预存4套坐标，rotation按模4更新，合法才提交。
- 为什么用一维棋盘？地址可用 $y \times 10 + x$ 计算，汇编实现简单且连续。
- 随机数是否真正随机？不是，是tick种子加LCG，足够演示但可改为7-bag。
- hlt会不会让游戏停止？不会，PIT和键盘中断会唤醒CPU。
- 当前主要局限？无wall kick、无等级加速、动作槽可能覆盖、测试未验证完整棋盘逻辑。

30秒推荐回答

我印象最深的是俄罗斯方块，因为它让我把操作系统底层能力真正用于一个持续交互应用。键盘动作通过IRQ1扫描码进入，中断只写动作标志；主循环被中断唤醒后进行碰撞检测和状态更新。自动下落使用PIT的tick，画面直接写0xB8000显存。棋盘是10×18数组，七种方块用四种旋转的坐标表表示，所有移动都遵循先验证再提交。它让我直观理解了操作系统怎样向应用提供输入、时间和显示服务。

一句代码索引

主循环14 → 键盘139 → 下落174 → 锁定185 → 生成208 → 碰撞233 → 坐标280 → 消行298 → 绘制340 → 数据487